2025 5th International Conference on Electrical, Computer and Energy Technologies (ICECET) | 979-8-3315-3559-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICECET63943.2025.11471909

Enhancing RAG Pipelines with Agent-Based Hallucination Detection and Guardrails Mechanisms

Vaibhav Mishra
Data Scientist
Genpact
Bengaluru, India
vaibhav.mishra@genpact.com

Vipul Sharma
Data Scientist
Genpact
Bengaluru, India
vipul.sharma2@genpact.com

Arjun Seth
Data Scientist
Genpact
Bengaluru, India
arjun.seth1@genpact.com

Prakash Selvakumar
Vice President
Genpact
Bengaluru, India
prakash.selvakumar@genpact.com

***Abstract*—Large Language Models (LLMs) have become increasingly more powerful and capable with time. By generating human-like text and understanding complex language patterns, they assist with tasks ranging from chatbots to content creation, revolutionizing how we interact with information. While they are very capable, they also face a significant challenge: hallucination. Hallucination refers to situations where LLMs generate factually inaccurate or misleading responses, veering off-the-rails from the truth. This research paper presents a comprehensive framework to address this issue, ensuring LLMs provide reliable and accurate information.**

The solution is twofold: implementing guardrails to keep LLMs on track and developing effective methods for hallucination detection. Two approaches are developed for the guard railing process and two more approaches are explored for the hallucination detection task. Each module will be explored in this research paper.

By combining the above methods, the framework ensures that responses are factually correct and thoroughly vetted through hallucination detection checks. Making it unique, robust, and novel, increasing the reliability of LLM-powered applications.

Keywords—Agents, Large Language Model (LLM), Hallucination, guardrails, Retrieval Augmented Generation (RAG), Finetuning, Knowledge Graphs.

I. INTRODUCTION

In the day and age of such advancements in the Large Language Model's (LLM) capabilities, multiple organizations and parties are interested in building custom chatbots to cater for their customers. The chatbot is required to follow a specific knowledge domain to respond to user queries.

This introduces the need for Retrieval-Augmented Generation (RAG) that utilizes a retriever to fetch the required context based on the user query, which is followed by a generator module, which uses LLMs to generate a relevant response for the user query based on the retrieved context.

In the RAG use case, which laid out the basis of this exploration was a RAG pipeline built for Apple 10K document for the fiscal year that ended on September 24, 2022. The document provides detailed financial information and performance analysis for investors and stakeholders.

One of the key challenges in deploying LLMs for real-world RAG use cases is their tendency to hallucinate [1]. Hallucination in the context of LLMs usually refers to a situation where the model generates content that is not based on factual or accurate information [2]. The occasional generation of outputs that appear plausible but are factually incorrect significantly undermines the reliability of LLMs in real-world scenarios. Similarly, there have been instances where the LLMs give off-the-rail responses plaguing user trust in the generative AI-powered applications. For example, in a bank chatbot, the LLM might give responses questioning the user's financial decisions, which is an unfortunate situation. To prevent from this occurring, guard rails are applied to the LLM, and it becomes of utmost importance for them to be adhered to.

This research paper presents a unique and comprehensive framework to address this issue, and it is twofold. Implementing guardrails to keep LLMs on track and developing effective methods for hallucination detection. Two approaches were explored for guard railing: The prompt concatenation method and the Langchain custom criteria evaluator, each offering unique advantages. Additionally, two powerful techniques for hallucination detection are also explored: a fine-tuned Tiny Language Model (TLM) and, Agentic RefChecker. The above explorations make the framework novel, robust and unique. Adding to the global effort of making responsible and ethical AI. They will be discussed in detail later in the paper.

The paper is divided into five sections: Introduction, Literature Review, Proposed Framework, Design of Experiments, and Results & Conclusion. The Literature Review covers related works. The Proposed Framework outlines the solution's architecture and its steps. Design of Experiments details dataset preparation and architecture specifics. Results & Conclusion highlight the benefits of this approach.

II. LITERATURE REVIEW

The survey [2] discusses the widespread issue of hallucination in large language models (LLMs), where models generate content that deviates from factual reality. It emphasizes the critical need for detection and mitigation strategies to address this challenge. For detection, the paper highlights methods like SelfCheckGPT [4], which identifies hallucinations

979-8-3315-3559-9/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on April 20,2026 at 17:29:35 UTC from IEEE Xplore. Restrictions apply.

without additional resources, and HaluEval [5], a benchmark for systematic assessment which is also utilized in this framework.

The paper [6] offers a broad look at ways to lessen hallucination in language models. It gives special attention to RAG pipelines, which is relevant to this research. The paper's exploration of RAG, knowledge retrieval, and CoNLI [7] provides useful knowledge for improving language model reliability and creating strong hallucination detection methods. The paper's detailed look at the challenges and classification of these techniques forms a strong base for further development of the framework, making it robust.

Various methods have been proposed to mitigate hallucination in RAG, including context injection and prompt engineering. RAGTruth [1] is a dataset specifically designed for hallucination detection which provides a comprehensive benchmark for evaluating the effectiveness of hallucination detection methods. The development of high-quality datasets and advanced methods for hallucination detection is essential for ensuring the reliability and trustworthiness of LLMs in real-world applications.

The approach mentioned in the paper addresses the limitations of LLMs, which often provide incorrect or outdated information due to their reliance on static training data. Particularly useful in scenarios where access to internal information enhances the accuracy and relevance of generated responses. More in-depth analysis can be found in this research paper. [8]

The hallucination evaluation benchmark for large language models (HaluEval) is a large collection of generated and human-annotated hallucinated samples for evaluating the performance of LLMs in recognizing hallucinations.[5]. The construction pipeline of HaluEval involves a two-stage approach, sampling-then-filtering, to generate diverse hallucinated samples. The benchmark can facilitate research in understanding what types of content and to which extent LLMs tend to hallucinate, ultimately paving the way for building more effective and dependable LLMs in the future.

Previous studies on hallucination detection have mainly concentrated on post-processing techniques for hallucination detection, which tend to be computationally intensive and limited in effectiveness due to their separation from the LLM's inference process. To overcome these limitations, MIND [9] is introduced, an unsupervised training framework that leverages the internal states of LLMs for real-time hallucination detection without requiring manual annotations. It processes the internal states of the LLMs using a simple MLP Layer for the classification task.

HELM [9], a new benchmark for evaluating hallucination detection across multiple LLMs, featuring 6 LLM outputs and their corresponding internal states during their inference process. MIND outperforms existing state-of-the-art methods in hallucination detection and gives a new perspective on LLM hallucination.

Guardrails [10] have emerged as a core safeguarding technology, filtering the inputs or outputs of LLMs to address issues like ethical use, data biases, privacy, and robustness. However, the complexity of LLMs and their closed-source nature present substantial challenges in developing effective guardrails. A systematic, multidisciplinary approach is required to carefully specify requirements, design sophisticated neural-symbolic solutions, and thoroughly verify and validate the guardrails. This is necessary to manage the inherent complexity of balancing multiple, potentially conflicting requirements like mitigating hallucinations, toxicity, fairness, and biases.

The paper [11] advanced hallucination detection with a multi-agent framework that uses specialized agents to identify and flag hallucinations in AI-generated content. Over 300 prompts designed to induce hallucinations were processed through the pipeline, where each agent identified unverified claims and clarified speculative content. Structured JSON messages ensured key information about hallucinations was preserved and shared between agents.

The paper proposed new Key Performance Indicators (KPIs) to measure hallucination levels, such as Factual Claim Density, Grounding References, Disclaimer Frequency, and Contextualization Score. These KPIs offered a clear way to assess detection effectiveness. Results showed the multi-agent framework reduced hallucinations while improving transparency and clarity in AI responses.

This structured approach strengthened the experimental design and reliability of findings. The impressive research presented in this paper backed the approach of choosing agents to solve the hallucination detection task, further validating the multi-agent framework's effectiveness in identifying and flagging hallucinations in AI-generated content.

III. PROPOSED FRAMEWORK

1. *Ensuring guard railing of LLM responses*

Appending Guardrail Prompt – A custom business-specific guardrail curated after in-depth research and multiple experiments can be appended to the original RAG Prompt.

This will ensure that more use-case-specific guardrails are being applied to the LLM system and not the very blatant ones that most LLMs already adhere to. The prompt is 540 tokens (OpenAI tokenizer - cl100k) thus does not take much of the context window and enhances the reliability of the LLM response without an additional LLM call.

Langchain Custom Evaluator – This approach is for the evaluation of the LLM response after it is generated by the RAG pipeline. The LLM response along with the business-specific guardrails is then sent to an LLM call for evaluation.

If any of the checks are failed then the response is regenerated, otherwise passed to the next step.

2. *Ensuring LLM Hallucination Detection*

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on April 20,2026 at 17:29:35 UTC from IEEE Xplore. Restrictions apply.

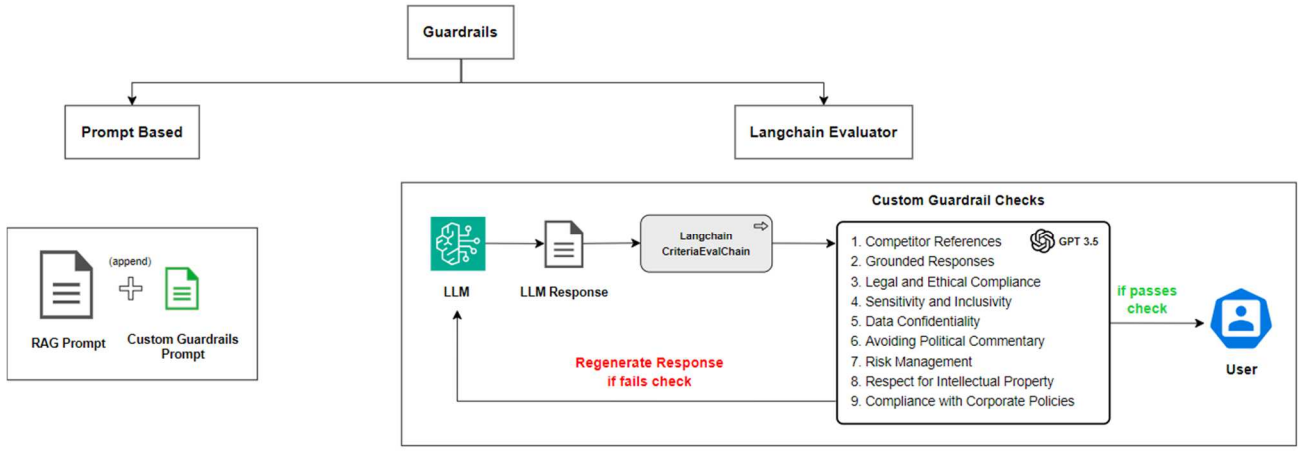


Fig 1. Guard Railing Architecture

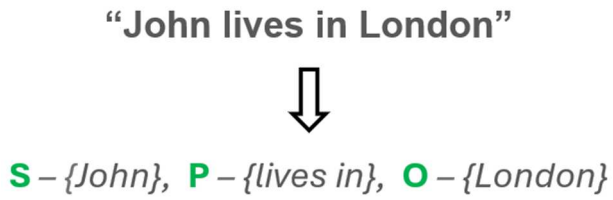


Fig 2. Knowledge Graph Triplet Example

- ***TLM Finetuning*** – After leveraging high-quality datasets like RAGTruth and HaluEval, a TLM is finetuned for hallucination detection tasks. Utilization of memory-efficient techniques was done like quantizing the model to 4-bit and using Unsloth for efficient memory management

  Once the fine-tuned model is ready, LLM responses can be given to the fine-tuned model for hallucination detection and a classification for the same can be obtained. Further nuances are mentioned in the design of experiment section.

- ***Agentic RefChecker*** – RefChecker was a solution built by Amazon where the LLM responses are firstly broken down into knowledge graph triplets (subject, predicate, object) as shown in Fig 2. Then the response triplets are compared with the user query + retrieved context and classified as "hallucinated" or "not hallucinated."

This methodology was updated for our solution with 4 major changes to optimize the results-

- Replacing the LLMs with Agents powered by TLMs finetuned specifically for each of the steps.
- Modifying the prompt completely for it to work with TLM Agents. The Amazon's prompt resulted in improper triplet generation and triplet comparison.
- Comparing the triplets individually with the query plus context and then giving a judgement. Based on each of the judgements the classification of the whole response will be done. This was done to minimize the load on the TLM. Because Amazon made use of powerful LLMs like Generative Pre-Trained Transformer (GPT) 4 and Claude 3 Sonnet, they did not have to include this step. An in-depth explanation is available in the next section.
- Lastly regex was applied to absolutely minimize the load on Agent, to classify the response as "hallucinated" or "not hallucinated" programmatically. Bigger models like GPT4 could easily handle the task of comparison and classification simultaneously, unlike the TLMs. Hence this step becomes necessary to extract the best performance out of TLMs in such a complex task.

IV. DESIGN OF EXPERIMENT

Following are the designs of experiments for each of the explorations individually-

A. Guardrails

After testing LLM behavior, nine guardrails (Fig. 1) were set to ensure safe, compliant responses with minimal tokens. A tailored business prompt was designed, and the guardrails can be customized as needed while keeping the core structure intact, offering flexibility in the process.

- *Appending Guardrail Prompt* – This approach prevents an extra LLM call for verification, decreasing the cost and resource utilization for the whole guard railing process.
- *Langchain Custom Evaluator* – This approach utilizes GPT 3.5 Turbo to evaluate LLM responses increasing the accuracy and the reliability of detection. Any LLM can be chosen for evaluation purposes considering the price and latency requirements.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on April 20,2026 at 17:29:35 UTC from IEEE Xplore. Restrictions apply.

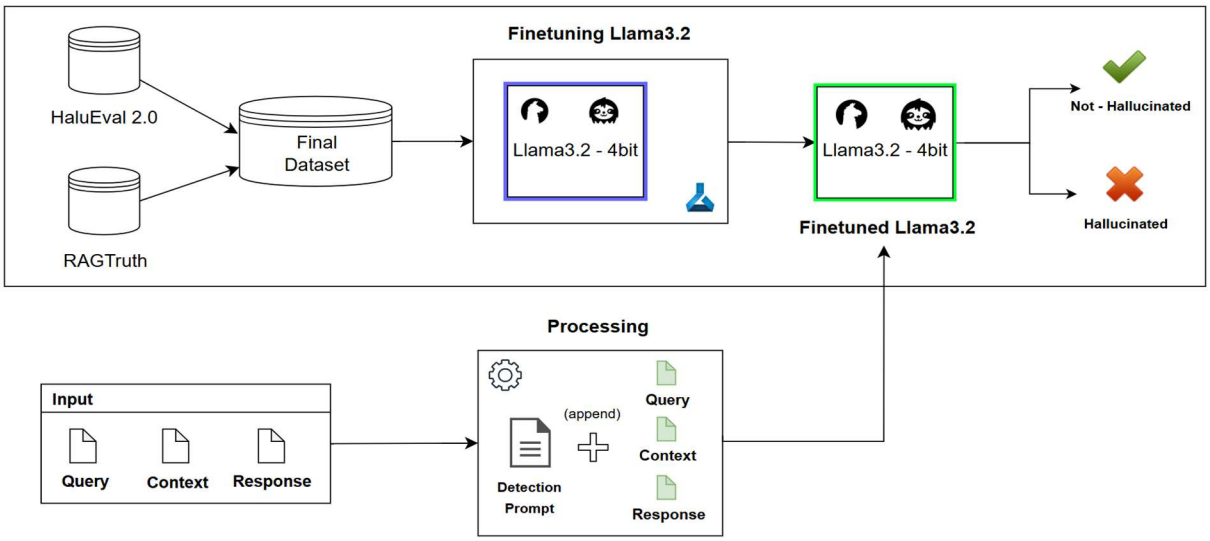


Fig. 3. Llama 3.2 Finetuning Architecture

B. *TLM Finetuning*

The training dataset contains 14,820 total samples with 7,410 hallucinated and not hallucinated samples each. The samples are collected from two of the most comprehensive hallucination detection datasets out there; RAGTruth and HaluEval. The distribution followed is shown in Fig 4. The details are as follows –

RAGTruth:

- Contains a total of 17,790 data points.
- *Summarization Datapoints*: 3,372 datapoints, with 1,686 labelled as hallucinated and 1,686 as not hallucinated.
- *Question and Answer Datapoints:* 3,448 datapoints, with 1,724 labelled as hallucinated and 1,724 as not hallucinated. Only QA and the summarization data points were picked because they matched the use case the most.
- Converted the summarization data points to QA format by randomly appending a question (from a list of questions) to the initial summarization query.

HaluEval:

- Contains a total of 35,000 data points.
- *Summarization Datapoints:* 10,000 datapoints, only 4,000 were chosen with 2,000 labelled as hallucinated and 2,000 as not hallucinated.
- *Question and Answer Datapoints:* 10,000 datapoints, only 4,000 were chosen with 2,000 labelled as hallucinated and 2,000 as not hallucinated.

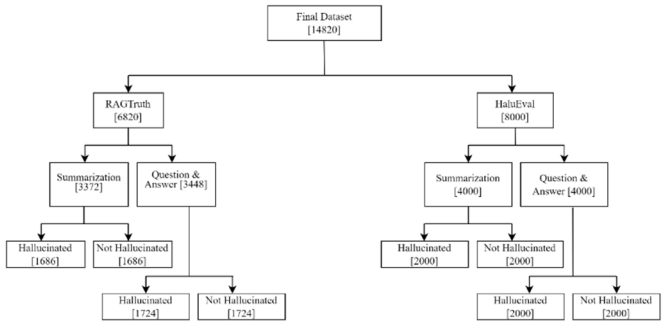


Fig. 4. Training Data Preparation

Data point in the final dataset consists of the following fields:

- *Query:* The initial prompt or question.
- *Context:* The surrounding text or background information relevant to the query.
- *Response:* The generated response to the query is based on the context.

The first finetuning of Llama 2 7B was performed for the task. Finetuning was done on compute mlg5.24xlarge provided by Amazon Web Services (AWS) with 4xA10G GPUs and 384 GB RAM. The model was loaded from Hugging Face.

Then finetuning of Mistral 7B was performed. The finetuning was done on Kaggle with 2xT4 GPUs and 30 GB RAM. The model was loaded from Hugging Face.

After which Llama 3 8B finetuning was done. The specific model used was an unsloth Llama 3 8B model quantized to 4 bits as shown in Fig 3. QLoRa (Quantized Low-Rank Adapters) was used as the optimization technique. The instance used to finetune the model had 1 Nvidia A100 GPU and 220 GB RAM, and it took ~5 hours to train.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on April 20,2026 at 17:29:35 UTC from IEEE Xplore. Restrictions apply.

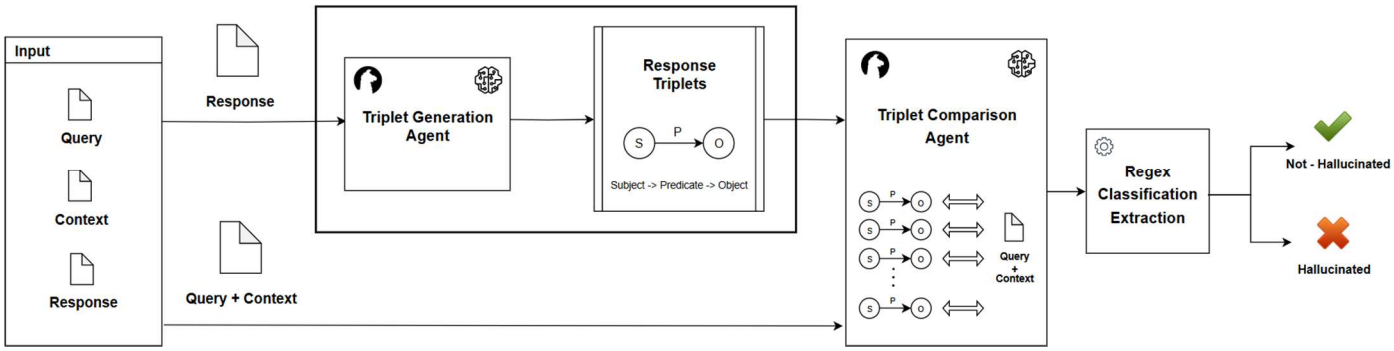


Fig. 5. Agentic RefChecker Architecture

Finally, the latest and more capable model Llama 3.2 3B model was finetuned on the same dataset. The model was quantized to 4 bits to make the training faster and optimize the usage of memory. The final optimized hyperparameters were as follows –

- Lora rank – 16, Lora alpha – 16, Lora dropout – 0.05,
- Batch size – 8, Training precision – fp16,
- Learning rate scheduler – cosine,
- Optimizer – paged_adam_8bit

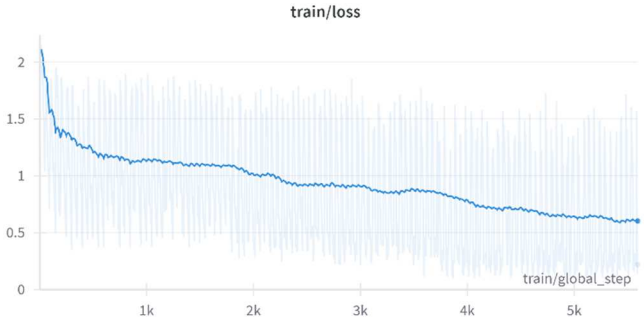


Fig. 6. Training Loss Graph vs Epochs

The instance used to finetune the model had 1 Nvidia A100 GPU and 220 GB RAM, and it took ~4 hours to train with the loss graph shown in Fig 6.

C. Agentic RefChecker –

In the finetuning TLM activity, it was noticed that there were some inconsistencies in the TLM's performance due to the nature of the input it was being fed. The input consisted of long and continuous RAG contexts and LLM responses. This gave an insight into the relationship between the nature of input sequence and the TLM's performance

To address this, knowledge graph concepts were introduced, as hallucination detection proved too complex for a simple TLM call. An agent-based approach was adopted as shown in Fig 5, splitting the task into two subtasks: triplet generation and triplet comparison. Both agents utilized a fine-tuned Llama 3.2 3B model with specialized tools.

1. *Triplet Generation Agent*

   Task: Break down LLM responses into concise Knowledge Graph (KG) triplets.

   Dataset Creation: - Data was sourced from [Quxiaolong2024/entity_extend_triples-large] and [Sefika/KGQA-data-triples] datasets on Hugging Face Hub, focusing on structure, size, and integration into a unified dataset for finetuning.

   A total of **7,000** samples were curated:

   - **3,500** simple 1 liner statements.
   - **3,500** consists of complex multiple liner statements.

   Tools:

   - *Entity Extraction:* Identifies named entities, including custom domain-specific ones.
   - *Relation Extraction***:** Extracts relationships, handling synonyms and paraphrased contexts.
   - *Triplet Validation & Deduplication:* Ensures logical coherence, and removes duplicates.

2. *Triplet Comparison Agent*

   Task: Compare generated triplets with the query and context for validation.

   Dataset Creation: The dataset was taken from the finetuning TLM task which had response-context pairs, where the natural language responses were decomposed into KG triplets using the GPT-4o model.

   Tools:

   - *Entity Matching:* Matches entities in triplets with context using similarity metrics or exact matching.
   - *Relation Matching:* Validates relationships by checking semantic alignment with the context.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on April 20,2026 at 17:29:35 UTC from IEEE Xplore. Restrictions apply.

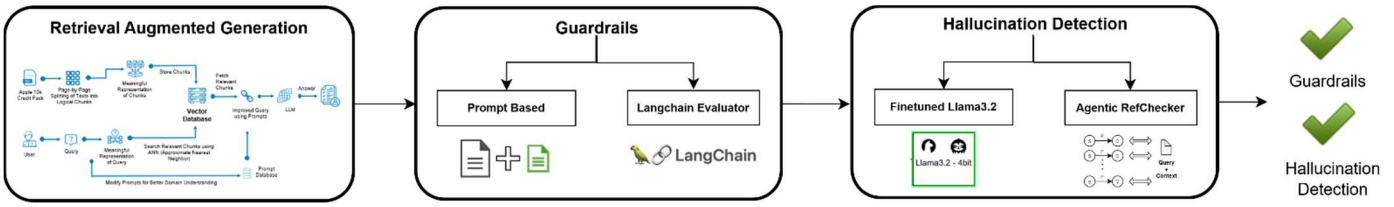


Fig 7. Final Architecture of the Framework

First, LLM responses are processed by the Triplet Generation Agent to create KG triplets. These are then passed to the Triplet Comparison Agent, which evaluates them against the query and context, categorizing each triplet as:

- *Correct* - The triplets contextually and semantically match with the given context, respectively.
- *Incorrect* - The triplets are not contextually and semantically adhering to the respective context.
- *Neutral* - The triplets have some knowledge inferred from the context and are not explicitly mentioned there.

V. RESULTS

A. TLM Finetuning Experimentation

TLM fine-tuning experiments showed improved classification performance. Llama 2 performed poorly with frequent hallucinations, while Mistral 7B was better but inconsistent. Llama 3 delivered reliable accuracy. Finally, Llama 3.2 3B achieved the highest accuracy (see Table 1).

TABLE I. FINETUNING RESULTS

Sno.	Model Name	Accuracy	Recall	Precision	F1-Score
1	Llama 2 7B	56.0%	0.0%	0.0%	0.0%
2	Mistral 7B	63.2%	65.7%	64.33%	64.4%
3	Llama 3 8B	75.0%	70.00%	77.83%	73.68%
4	**Llama 3.2 3B**	**79.16%**	**75.47%**	**83.33%**	**79.21%**

B. Agentic RefChecker

The results obtained for the Agentic RefChecker exploration are mentioned in Table 2. The results have improved significantly compared to the fine-tuning process.

TABLE II. AGENTIC REFCHECKER RESULTS

Sno.	Model Name	Accuracy	Recall	Precision	F1-Score
1	**Llama 3.2 3B**	**87.1%**	**86.02 %**	**88.00%**	**87.00%**

VI. CONCLUSION

This research paper introduces a robust framework for detecting hallucinations in RAG pipelines, combining customized Guardrails for compliance, Agentic RefChecker using knowledge graph triplets, and Agentic TLM fine-tuning to boost classification accuracy. The proposed approach significantly enhances reliability, advancing trustworthy and effective generative AI applications in real-world scenarios.

ACKNOWLEDGMENT

Genpact India Pvt. Ltd. supported this project.

REFERENCES

[1] C. Niu et al., "RAGTRUTH: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models," arXiv.org, Dec. 31, 2023. https://arxiv.org/abs/2401.00396

[2] V. Rawte, A. Sheth, and A. Das, "A survey of hallucination in large foundation models," arXiv.org, Sep. 12, 2023. https://arxiv.org/abs/2309.05922

[3] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12):1–38.

[4] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," arXiv.org, Mar. 15, 2023. https://arxiv.org/abs/2303.08896

[5] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "HalUEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models," arXiv.org, May 19, 2023. https://arxiv.org/abs/2305.11747

[6] S. M. T. I. Tonmoy et al., "A comprehensive survey of hallucination mitigation techniques in large language models," arXiv.org, Jan. 02, 2024. https://arxiv.org/abs/2401.01313

[7] D. Lei et al., "Chain of natural language inference for reducing large language model ungrounded hallucinations," arXiv.org, Oct. 06, 2023. https://arxiv.org/abs/2310.03951

[8] J. Li, Y. Yuan, and Z. Zhang, "Enhancing LLM Factual Accuracy with RAG to Counter Hallucinations: A Case Study on Domain-Specific Queries in Private Knowledge-Bases," arXiv.org, Mar. 15, 2024. https://arxiv.org/abs/2403.10446

[9] W. Su et al., "Unsupervised Real-Time Hallucination Detection based on the Internal States of Large Language Models," arXiv.org, Mar. 11, 2024. https://arxiv.org/abs/2403.06448

[10] Y. Dong et al., "Building guardrails for large language models," arXiv.org, Feb. 02, 2024. https://arxiv.org/abs/2402.01822

[11] Diego Gosmar and Deborah Dahl, "Hallucination Mitigation Using Agentic Ai Natural Language-Based Frameworks" 2501.13946, Jan. 19, 2025.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on April 20,2026 at 17:29:35 UTC from IEEE Xplore. Restrictions apply.